AS4SAN_Workshop_DataLad

July 17, 2026

0.1 Reproducible data management for big neuroimaging

0.1.1 Where does the data come from?

AS4SAN Workshop

Arush Arun — Data Pipelines Fellow, NIF, UQ

github.com/arush-arun/AS4SAN-Workshop

0.2 Every other module assumes the data is already here

> *"Download the dataset and place it in `./data/`"*
>
> — Every tutorial ever

- Preprocessing pipelines assume the files exist
- Analysis scripts assume a specific directory layout
- Papers reference "the dataset" but not *which version*

Nobody talks about how the data got there — or whether it's the same data you had last week.

0.3 Does this look familiar?

```
my_study/
  data_final/
  data_final_v2/
  data_FINAL_really/
  analysis_old.R
  analysis_new.R
  analysis_new_fixed.R
  results_fig1_BACKUP.png
  README_ignore_this.txt
```

- Which version of the data produced which figure?
- What changed between "final" and "final_v2"?
- Could a colleague reproduce your analysis from this folder?

Two things can save you here: 1. **Version control** (git) — tracks every change so you never need `_final_v2` again 2. **Standardization** (e.g. BIDS) — gives you a predictable, consistent folder layout like `sub-01/anat/`, `sub-01/func/` so everyone organizes data the same way

0.4 A quick intro to git

git is a tool that tracks changes to files over time — like "track changes" in Word, but for entire folders.

What git gives you: - A complete history of every change - The ability to go back to any previous version - A record of *who* changed *what* and *when* - Safe collaboration — no overwriting each other's work (e.g. two people edit `analysis.R` on separate branches, then git merges both changes together)

The basic cycle:

```
# 1. Make changes to your files

# 2. Stage - choose what to save
git add my_analysis.R

# 3. Commit - save a snapshot
git commit -m "Fix outlier filter"

# 4. Push - share with collaborators
git push
```

0.5 With git, that same project looks like this

```
my_study/
   .git/                    ← git tracks the full history here
   data/
   analysis.R
   results_fig1.png
   README.md
```

No more `_final_v2`, `_old`, `_BACKUP` — git keeps every version for you:

```
$ git log --oneline   # show the commit history - one line per snapshot
a3f1d2c  Fix outlier filter in analysis
b7e4a91  Add figure 1
c9d0e83  Clean raw data
f1a2b3c  Initial commit
```

- One file, one name — the history lives in git, not in the filename
- Need last week's version? `git checkout b7e4a91` — this *temporarily* visits that snapshot (like flipping back through a notebook), `git checkout master` brings you back to the latest
- Need to see what changed? `git diff`

0.6 The concrete problem: MR-RATE

Patients	83,425
MRI volumes	705,254
Total size	8.1 TB
Modalities	T1, T2, FLAIR, SWI, MRA

Extras	radiology reports, brain masks, segmentations

You can't just download 8 TB.

- Too large for a single machine
- Evolving — new releases, corrections
- Multi-site (Zurich, Istanbul, NVIDIA)
- You only need a *subset* for your analysis

Hugging Face: `Forithmus/MR-RATE` *— CC BY-NC-SA 4.0*

0.7 git is great — but it wasn't built for big data

git handles well	git struggles with
Analysis scripts (R, Python)	MRI scans (hundreds of MB each)
Participant lists, metadata (CSV, TSV)	Large datasets (GBs to TBs)
Documentation and notes	Binary files that don't "diff" well
Configuration files	Data stored on remote servers

We need something that works *like* git, but can handle big neuroimaging data.

That's where DataLad comes in.

0.8 DataLad: version control that works for data

- **git** handles the small stuff (scripts, metadata, history)
- **git-annex** handles the big stuff (scans, volumes) — stores *pointers*, not the actual files
- **DataLad** wraps both so you don't need to think about the layers underneath

0.9 The core loop: install → get → run → rerun

Step	Command	What happens
install	`datalad install -s <url>`	Clone metadata — no large files yet
get	`datalad get sub-01/anat/*.nii.gz`	Fetch only the files you need
run	`datalad run -m "msg" -- <cmd>`	Execute & record what you did
rerun	`datalad rerun`	Reproduce from the recorded history

Let's try it live!

---

1 Live demo: the DataLad core loop

We'll walk through the four steps using a real dataset from OpenNeuro.

1.0.1 Step 0: Install DataLad and git-annex

```
[ ]: # Install DataLad and git-annex
!pip install -q datalad datalad-installer
!apt-get install -y -qq netbase > /dev/null 2>&1
!datalad-installer git-annex -m datalad/git-annex:release

# Configure git (required by DataLad)
!git config --global user.name "Workshop User"
!git config --global user.email "workshop@example.com"

# Verify installation
!datalad --version
!git-annex version --raw
```

1.0.2 Step 1: `datalad install` — clone metadata only

This clones the full directory tree and metadata, but **no large files**. It's fast even for terabyte-scale datasets.

We'll use a dataset from OpenNeuro — a free platform hosting 1,000+ neuroimaging datasets, all formatted in BIDS and accessible via DataLad. This dataset (ds000101) follows the **BIDS standard** (Brain Imaging Data Structure) — a community convention for organizing neuroimaging files into a predictable `sub-XX/anat/`, `sub-XX/func/` layout. BIDS and DataLad are a natural pair: BIDS standardizes *how files are named*, DataLad handles *how they're versioned and shared*.

```
[ ]: !datalad install -s https://github.com/OpenNeuroDatasets/ds000101.git␣
  ↪ds000101-demo


# Look at the directory structure - all the files are listed,
# but the large ones haven't been downloaded yet
import os
os.chdir('ds000101-demo')
!ls -la sub-01/anat/

#Notice the file is a symlink - the actual data isn't here yet. DataLad knows␣
  ↪*about* the file (its size, checksum, where to get it) but hasn't downloaded␣
  ↪the content.
```

1.0.3 Step 2: `datalad get` — fetch only the files you need

Instead of downloading the entire dataset, we fetch just one file.

```
[ ]: !datalad get sub-01/anat/sub-01_T1w.nii.gz
# Now the file is real - check its actual size (-L follows the symlink)
# git-annex stores files as symlinks to .git/annex/objects/ - the hash
# in the filename IS the checksum, guaranteeing data integrity
!ls -lhL sub-01/anat/sub-01_T1w.nii.gz
```

1.0.4 Step 3: `datalad run` — execute and record provenance

First, let's install nilearn so we can do a real neuroimaging analysis. Then we'll wrap it with `datalad run` so the provenance is automatically recorded.

[ ]:
```
!pip install -q nilearn
```

We'll write a small Python script that computes a brain mask from the T1w image and saves a plot. Then `datalad run` will execute it and record exactly what happened.

[ ]:
```
%%writefile /content/compute_brain_mask.py
"""Compute a brain mask from a T1w image and save a figure."""
import sys
from nilearn.masking import compute_epi_mask
from nilearn import plotting
import nibabel as nib

t1_path = sys.argv[1]
output_path = sys.argv[2]

# Compute brain mask
img = nib.load(t1_path)
mask = compute_epi_mask(img)

# Save the plot - ortho mode needs (x, y, z) coordinates
fig = plotting.plot_anat(img, display_mode='ortho',
                         title='T1w with brain mask overlay')
fig.add_contours(mask, colors='r', linewidths=0.5)
fig.savefig(output_path, dpi=150)
fig.close()

print(f"Brain mask computed and figure saved to {output_path}")
```

[ ]:
```
# Run the analysis wrapped by DataLad - provenance is recorded automatically
!datalad run -m "Compute brain mask and save overlay plot" \
  --input sub-01/anat/sub-01_T1w.nii.gz \
  --output brain_mask_overlay.png \
  -- python /content/compute_brain_mask.py sub-01/anat/sub-01_T1w.nii.gz␣
 ↪brain_mask_overlay.png
```

[ ]:
```
# Display the result
from IPython.display import Image, display
display(Image('brain_mask_overlay.png'))
```

[ ]:
```
# The provenance is recorded in git history - DataLad knows exactly
# what command produced brain_mask_overlay.png and from which input
!git log --oneline -3
```

1.0.5 Step 4: `datalad rerun` — reproduce from recorded provenance

`datalad rerun` reads the git history, finds the last `datalad run` commit, and **replays the exact same command** — same inputs, same code, same outputs.

What it does under the hood: 1. Looks up the recorded command from the git log 2. Ensures all declared `--input` files are available (auto-fetches if needed) 3. Unlocks the `--output` files so they can be overwritten 4. Re-executes the original command 5. Commits the result

Why this matters: - A reviewer asks *"can you reproduce figure 3?"* — you run `datalad rerun` and it's done - A collaborator clones your dataset and runs `datalad rerun` — they get the same result, without needing to know what command you originally ran - If the data changes (e.g. a corrected scan), rerun will use the updated input and you can compare outputs

The real-world workflow:

```
# A collaborator on a different machine:
datalad install -s <your-dataset-url> the-project
cd the-project
datalad rerun   # reproduces your analysis - two commands, full provenance
```

[ ]:
```
!datalad rerun
```

[ ]:
```
# Same figure, reproduced from provenance!
display(Image('brain_mask_overlay.png'))
```

[ ]:
```
# Let's look at exactly what DataLad recorded - the full provenance
# This is what makes rerun possible: every detail is in the git history
!git log -1 --format="%B" HEAD~1
```

The commit message contains a `=== Do not change lines below ===` block — that's the machine-readable provenance record. It stores the exact command, inputs, outputs, and their checksums. This is what `datalad rerun` reads to reproduce your work.

1.0.6 Bonus commands: `datalad status` and `datalad drop`

Two more commands worth knowing:

- **`datalad status`** — like `git status`, but also shows which large files have content locally vs. which are just pointers
- **`datalad drop`** — the opposite of `get`. Removes the local copy of a file **but keeps the pointer**, so you can `datalad get` it again anytime

This is what makes DataLad practical for big data — you fetch what you need, work with it, and then free up disk space without losing track of anything.

[ ]:
```
# Check what DataLad knows about our files
# If this says "nothing to save, working tree clean" - that's good!
# It means `datalad run` already committed everything automatically.
!datalad status
```

```
[ ]: # Drop the T1w scan - frees disk space, but DataLad remembers where to get it
     !datalad drop sub-01/anat/sub-01_T1w.nii.gz

     # The file is back to being a symlink - pointer only, no content
     !ls -lh sub-01/anat/sub-01_T1w.nii.gz

     # Need it again? Just `datalad get` it - same command as before
     # !datalad get sub-01/anat/sub-01_T1w.nii.gz
```

1.1 Why it matters

1.1.1 Provenance — your digital lab notebook

"Which version of which data produced this figure?"

DataLad records the **exact command**, **inputs**, and **outputs** — automatically.

1.1.2 Selective access — download only what you need

Get 50 MB of scans for your analysis, not 8 TB for the whole dataset.

1.1.3 Collaboration — no more emailing files

Everyone works from the same dataset. The history shows who changed what and when.

1.2 "But I work with patient data — can I still use this?"

Yes. DataLad doesn't require you to share data publicly. Here's how to keep things confidential:

What git/DataLad sees vs. what stays private:

Tracked (in git history)	NOT tracked
File *names* (e.g. `sub-01/anat/sub-01_T1w.nii.gz`)	File *contents* (the actual scan)
Checksums (a hash, not the data)	Patient names, dates of birth, etc.
Commands you ran (`datalad run ...`)	Anything in `.gitignore`

Practical steps:

1. **De-identify first** — use BIDS-style anonymous IDs (`sub-01`, `sub-02`), never real names in filenames
2. **Keep the dataset private** — use a private GitHub/GitLab repo, or an institutional server. DataLad works with any git remote
3. **Exclude sensitive files** — add consent forms, clinical notes, or raw DICOM headers to `.gitignore`
4. **Deface scans** — wrap defacing with `datalad run` so even that step is reproducible:

```
datalad run -m "Deface T1w scans" \
  --input sub-01/anat/sub-01_T1w.nii.gz \
  --output sub-01/anat/sub-01_T1w.nii.gz \
  -- pydeface sub-01/anat/sub-01_T1w.nii.gz
```

The key insight: DataLad gives you reproducibility and version control *without* requiring you to make anything public. You control who has access — DataLad just makes sure everyone with access is working from the same versioned dataset.

1.3 You can also access preprocessed data

So far we've worked with **raw** scans. But what if someone has already run a preprocessing pipeline and shared the results?

The Human Connectome Project (HCP) distributes preprocessed fMRI data — and it's available as a **DataLad dataset**. You can browse 400+ subjects' working memory task results and fetch only the participants you need.

[ ]:
```
# Go back to the root directory
import os
os.chdir('/content')

# Install the HCP working memory preprocessed dataset (metadata only - fast)
!datalad install -s https://github.com/datalad-datasets/hcp_wm_preprocessed.git␣
 ↪hcp_wm_preprocessed
os.chdir('/content/hcp_wm_preprocessed')

# See the available subjects - each folder is one participant
!ls -d */ | head -10
!echo "..."
!ls -d */ | wc -l
print("subjects available")
```

[ ]:
```
# Look inside one subject - preprocessed results in MNI space
!ls 100206/MNINonLinear/Results/
```

[ ]:
```
# Fetch just one subject's working memory results - not the entire dataset!
!datalad get 100206/MNINonLinear/Results/tfMRI_WM_LR/
!ls -lhL 100206/MNINonLinear/Results/tfMRI_WM_LR/
```

Same `install` → `get` workflow, but now you're accessing **preprocessed data** that someone else already ran through the HCP pipeline. No need to re-run preprocessing yourself — just grab the results you need and start your analysis.

This is the power of combining DataLad with shared preprocessing: reproducible access to analysis-ready data, one subject at a time.

Bonus: accessing MR-RATE via Hugging Face (click to expand)

MR-RATE isn't a DataLad dataset — it's hosted on Hugging Face as a **gated dataset**. You need to:

1. Create a Hugging Face account
2. Go to the MR-RATE dataset page and **request access**
3. Once approved, generate an access token

The same principle applies — browse the metadata first, download only what you need. The tools are different but the idea is the same.

```
!pip install -q huggingface_hub
from huggingface_hub import login
login(add_to_git_credential=False)

from huggingface_hub import HfApi
api = HfApi()
files = api.list_repo_files("Forithmus/MR-RATE", repo_type="dataset")
print(f"Total files: {len(files)}")

from huggingface_hub import hf_hub_download
path = hf_hub_download("Forithmus/MR-RATE", filename="metadata/batch00_metadata.csv", repo_typ
```

1.4 Pointers and resources

Resource	Link
DataLad Handbook	handbook.datalad.org
BIDS Standard	bids.neuroimaging.io
OpenNeuro	openneuro.org
DataLad datasets	datasets.datalad.org
HCP Preprocessed (DataLad)	github.com/datalad-datasets/hcp_wm_preprocessed
MR-RATE	huggingface.co/datasets/Forithmus/MR-RATE
This notebook	github.com/arush-arun/AS4SAN-Workshop

The DataLad Handbook is the single best starting point — it's a full tutorial with worked examples, no programming background required.

Next step — pipeline automation: DataLad's `datalad containers-run` command wraps containerized pipelines (e.g. fMRIPrep, MRIQC) so that the *entire processing environment* is recorded alongside inputs and outputs. Same provenance tracking as `datalad run`, but fully reproducible across machines. See the DataLad Handbook chapter on containers.

Citation: Halchenko et al. (2021). DataLad: distributed system for joint management of code, data, and their relationship. *Journal of Open Source Software*, 6(63), 3262. doi:10.21105/joss.03262

1.5 What should you remember?

- Version your datasets — not just your code
- Download only what you need — not the whole 8 TB
- Track provenance automatically — know exactly what produced each result
- Reproduce analyses — `datalad rerun`, done
- Share datasets without duplicating files — collaborators get pointers, not copies
- Standardize your layout with **BIDS** — DataLad works best when file organization is predictable
- Scale up with `datalad containers-run` — wrap full pipelines (fMRIPrep, MRIQC) for end-to-end reproducibility

---

2 Questions?

2.1 Thank you!

Thanks to **Lena** and the team members for organising.

Supported by **National Imaging Facility (NIF)** and **The University of Queensland (UQ)**.

Clinical Neuroimaging Biomarkers Lab

clinical-neuroimaging-biomarkers-lab.github.io